HDOC Day-66 Activity

Recursive Multiplication of Two Square Matrices

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-66

Aim

To write a C program that multiplies two square matrices of order n using **recursion** for the multiplication logic. Iterative loops are not used to calculate the product matrix.

Problem Requirements

- Read the order n of two square matrices.
- Read the elements of Matrix A and Matrix B.
- Multiply A and B to obtain Matrix C.
- Use recursion for calculating every element of the product matrix.
- Do not use iterative loops for the multiplication logic.

Matrix Multiplication Rule

Each element C[i][j] is obtained by multiplying corresponding elements of row i of A and column j of B and adding them. For an n x n matrix, exactly n products contribute to each output element.

Recursive Approach

The program uses one recursive function to calculate the dot product for a single output position and another recursive function to move through every row and column of the result matrix. Thus, the multiplication itself contains no iterative loop.

Algorithm

1. Read n and the two n x n matrices.
2. For result position (row, col), recursively calculate A[row][0]*B[0][col] + ... + A[row][n-1]*B[n-1][col].
3. The dot-product base case occurs when k == n, returning 0.
4. Store the recursive dot-product result in C[row][col].
5. Recursively move to the next column.
6. When the last column is completed, recursively move to the first column of the next row.
7. Stop when row == n.
8. Display the resulting matrix.

C Program

```
#include <stdio.h>

int dot_product(int n, int A[n][n], int B[n][n],
                int row, int col, int k)
{
    if (k == n)
        return 0;

    return A[row][k] * B[k][col] +
           dot_product(n, A, B, row, col, k + 1);
}

void multiply_recursive(int n, int A[n][n], int B[n][n],
                        int C[n][n], int row, int col)
{
    if (row == n)
        return;

    C[row][col] = dot_product(n, A, B, row, col, 0);

    if (col + 1 < n)
        multiply_recursive(n, A, B, C, row, col + 1);
    else
        multiply_recursive(n, A, B, C, row + 1, 0);
}

int main()
{
    int n;

    printf("Enter order of square matrices: ");
    scanf("%d", &n);

    int A[n][n], B[n][n], C[n][n];

    printf("Enter Matrix A:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &A[i][j]);

    printf("Enter Matrix B:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &B[i][j]);

    multiply_recursive(n, A, B, C, 0, 0);

    printf("Product Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            printf("%d ", C[i][j]);
        printf("\n");
    }

    return 0;
}
```

Important Note

The **for loops in main()** are used only to read and display matrices. The actual matrix multiplication is performed entirely by the recursive functions **dot_product()** and **multiply_recursive()**, satisfying the requirement that iterative loops are not used for multiplication logic.

Dry Run - 2 x 2 Example

Result element	Recursive calculation	Value

C[0][0]	1 x 5 + 2 x 7	19
C[0][1]	1 x 6 + 2 x 8	22
C[1][0]	3 x 5 + 4 x 7	43
C[1][1]	3 x 6 + 4 x 8	50

For A = [[1, 2], [3, 4]] and B = [[5, 6], [7, 8]], the product matrix is **[[19, 22], [43, 50]]**.

How the Recursion Works

Function	Purpose
dot_product()	Recursively moves k from 0 to n-1 and calculates one row-column dot product.
multiply_recursive()	Recursively moves across all columns and rows of the result matrix.
Base case of dot_product	k == n: no more terms remain, so return 0.
Base case of multiplication	row == n: all result elements have been calculated.

Complexity

Time Complexity: $O(n^3)$. There are n^2 output elements and each one requires n multiplications/additions.

Auxiliary Recursion Space: O(n) for the dot-product recursion depth. The result matrix itself requires $O(n^2)$ storage.

Result

The program successfully multiplies two square matrices. The complete multiplication logic is implemented recursively, while loops are used only for input and output.

Execution Example

```
Enter order of square matrices: 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
5 6
7 8
Product Matrix:
19 22
43 50
```